MINI PROJECT

STUDENT PERFORMANCE ANALYSIS

Department of Computer Science and Engineering

(Artificial Intelligence and Machine Learning)

Coimbatore Institute of Engineering and Technology

Name : Adhithyan J

Register Number : 71052510001

Subject Name: Data Science

Course Code: U23AMT01

Year/Semester: [II / III]

Date: 15.10.2026

INTRODUCTION:

The Student Performance Analysis project is a data-analysis system developed using Python, Pandas, Matplotlib, Seaborn and Scikit-learn to identify the major factors that influence the final grades of secondary-school students. It uses the UCI Student Performance Dataset, which contains demographic, social, family and school-related details of students from two Portuguese schools.

The system cleans the data, explores it through statistical summaries and visualizations, and studies how attributes such as study time, absences, past failures, parental education and lifestyle habits relate to the final grade (G3). Several regression models are then trained to predict the final grade and are compared using standard error metrics.

This project aims to help teachers and institutions understand what drives academic performance, so that students who need support can be identified early.

OBJECTIVES:

- To clean and preprocess the UCI Student Performance Dataset for analysis.
- To explore the relationship of study time, absences, past failures and family background with the final grade.
- To visualize grade distribution, correlations and group-wise performance using charts.
- To engineer new features such as average alcohol consumption, parental education and social index.
- To build and compare Linear, Ridge, Lasso, Decision Tree and Random Forest regression models.
- To evaluate the models using MAE, RMSE, R^2 and cross-validation.
- To identify the most important factors influencing students' final grades.

TOOLS AND TECHNOLOGIES:

- **Programming Language:** Python 3
- **Data Analysis Libraries:** NumPy, Pandas
- **Visualization Libraries:** Matplotlib, Seaborn
- **Machine Learning Library:** Scikit-learn
- **Development Environment:** Jupyter Notebook / Google Colab
- **Dataset:** UCI Student Performance Dataset (student-mat.csv)
- **Dataset Source:** UCI Machine Learning Repository
- **Platform:** Windows Operating System
- **Application Type:** Data Analysis and Prediction Syste

DATASET DESCRIPTION:

The dataset was collected from two Portuguese secondary schools using school reports and questionnaires. The Mathematics file (student-mat.csv) is used in this project. Each record has 33 attributes. The target variable is G3, the final grade on a scale of 0 to 20.

- **Demographic:** school, sex, age, address, famsize, Pstatus
- **Family background:** Medu, Fedu, Mjob, Fjob, guardian, famrel, famsup
- **Study related:** studytime, failures, schoolsup, paid, higher, absences, traveltime
- **Lifestyle:** activities, nursery, internet, romantic, freetime, goout, Dalc, Walc, health
- **Grades:** G1 (first period), G2 (second period), G3 (final, target)

PROGRAM CODE AND OUTPUT (INPUT & OUTPUT):

Step 1: Import Libraries and Load the Dataset

INPUT:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

sns.set_theme(style="whitegrid")

# Dataset: UCI Student Performance (student-mat.csv), semicolon separated
df = pd.read_csv("student-mat.csv", sep=";")

print("Shape of dataset:", df.shape)
print(df.head())
```

OUTPUT:

```
Shape of dataset: (395, 33)
  school sex  age address famsize Pstatus  ...  Walc  health absences  G1  G2  G3
0     GP   F   18       U     GT3       A  ...     1       3        6   5   6   6
1     GP   F   17       U     GT3       T  ...     1       3        4   5   5   6
2     GP   F   15       U     LE3       T  ...     3       3       10   7   8  10
3     GP   F   15       U     GT3       T  ...     1       5        2  15  14  15
4     GP   F   16       U     GT3       T  ...     2       5        4   6  10  10

[5 rows x 33 columns]
```

The dataset has 395 student records and 33 attributes. The preview shows the first five students with their demographic, family, study and grade details, confirming that the CSV file was loaded correctly.

Step 2: Dataset Information

INPUT:

```
df.info()
```

OUTPUT:

```
<class 'pandas.DataFrame'>
RangeIndex: 395 entries, 0 to 394
Data columns (total 33 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   school      395 non-null    str
 1   sex         395 non-null    str
 2   age         395 non-null    int64
 3   address     395 non-null    str
 4   famsize     395 non-null    str
 5   Pstatus     395 non-null    str
 6   Medu        395 non-null    int64
 7   Fedu        395 non-null    int64
 8   Mjob        395 non-null    str
 9   Fjob        395 non-null    str
 10  reason      395 non-null    str
 11  guardian    395 non-null    str
 12  traveltime  395 non-null    int64
 13  studytime   395 non-null    int64
 14  failures    395 non-null    int64
 15  schoolsup   395 non-null    str
 16  famsup      395 non-null    str
 17  paid        395 non-null    str
 18  activities  395 non-null    str
 19  nursery     395 non-null    str
 20  higher      395 non-null    str
 21  internet    395 non-null    str
 22  romantic    395 non-null    str
 23  famrel      395 non-null    int64
 24  freetime    395 non-null    int64
 25  goout       395 non-null    int64
 26  Dalc        395 non-null    int64
 27  Walc        395 non-null    int64
 28  health      395 non-null    int64
 29  absences    395 non-null    int64
 30  G1          395 non-null    int64
 31  G2          395 non-null    int64
 32  G3          395 non-null    int64
dtypes: int64(16), str(17)
memory usage: 102.0 KB
```

All 33 columns have 395 non-null values, so there are no missing entries. Sixteen attributes are numeric (age, Medu, studytime, failures, absences, G1, G2, G3 and others) and seventeen are text categories such as school, sex and Mjob.

Step 3: Summary Statistics

INPUT:

print(df.describe())

OUTPUT:

```
              age        Medu        Fedu  ...          G1          G2          G3
count  395.000000  395.000000  395.000000  ...  395.000000  395.000000  395.000000
mean    16.696203    2.749367    2.521519  ...   10.908861   10.713924   10.415190
std      1.276043    1.094735    1.088201  ...    3.319195    3.761505    4.581443
min     15.000000    0.000000    0.000000  ...    3.000000    0.000000    0.000000
25%     16.000000    2.000000    2.000000  ...    8.000000    9.000000    8.000000
50%     17.000000    3.000000    2.000000  ...   11.000000   11.000000   11.000000
75%     18.000000    4.000000    3.000000  ...   13.000000   13.000000   14.000000
max     22.000000    4.000000    4.000000  ...   19.000000   19.000000   20.000000

[8 rows x 16 columns]
```

The final grade G3 has a mean of 10.42 and a standard deviation of 4.58, with values from 0 to 20. The first and second period grades have similar means (10.91 for G1 and 10.71 for G2), so grades stay roughly stable across the year.

Step 4: Data Cleaning and Preprocessing

INPUT:

```
# Missing values and duplicates
print("Total missing values:", df.isnull().sum().sum())
print("Duplicate rows:", df.duplicated().sum())
df = df.drop_duplicates()

# Students who scored 0 in the final grade (G3)
print("Students with G3 == 0:", (df["G3"] == 0).sum())

# Outlier check on absences using the IQR rule
q1, q3 = df["absences"].quantile([0.25, 0.75])
upper = q3 + 1.5 * (q3 - q1)
print("Upper limit for absences:", upper)
print("Absence outliers:", (df["absences"] > upper).sum())

# Do students with G3 == 0 differ in absences?
zero = df[df["G3"] == 0]
nonzero = df[df["G3"] > 0]
print("Mean absences (G3 == 0):", round(zero["absences"].mean(), 2))
print("Mean absences (G3 > 0):", round(nonzero["absences"].mean(), 2))
print("Corr(absences, G3) for G3 > 0:", round(nonzero["absences"].corr(nonzero["G3"]), 3))
```

OUTPUT:

```
Total missing values: 0
Duplicate rows: 0
Students with G3 == 0: 38
Upper limit for absences: 20.0
Absence outliers: 15
Mean absences (G3 == 0): 0.0
Mean absences (G3 > 0): 6.32
Corr(absences, G3) for G3 > 0: -0.213
```

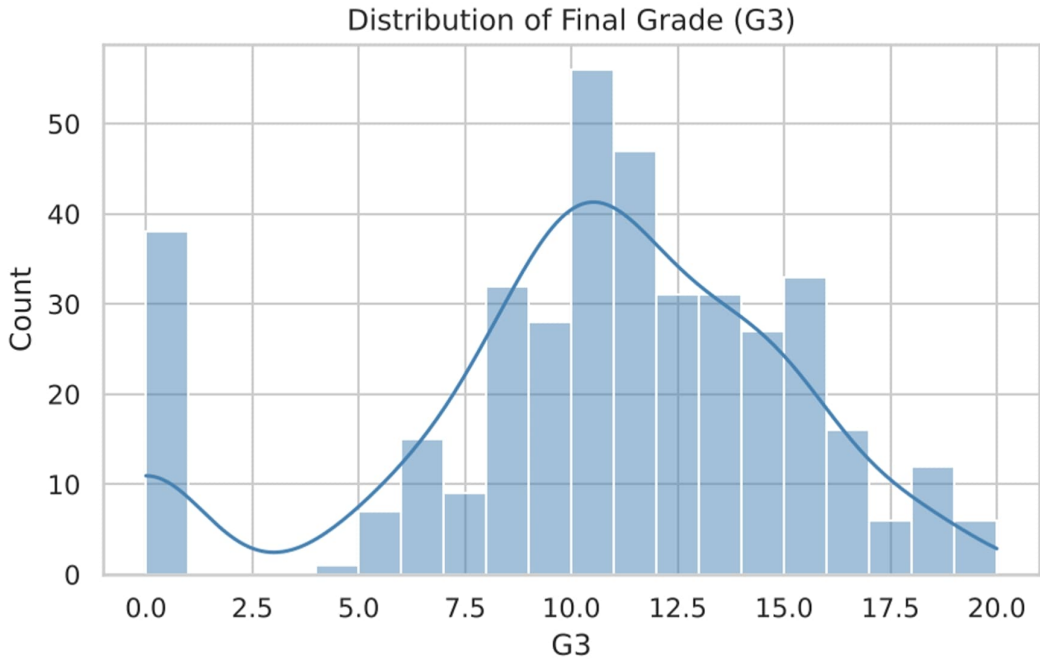
There are no missing values and no duplicate rows. Thirty-eight students have a final grade of 0, and 15 students are absence outliers above the IQR limit of 20. All 38 zero-grade students have 0 recorded absences, so absences behave differently for them: for students with G3 above 0 the correlation between absences and G3 is -0.21.

Step 5: Final Grade Distribution

INPUT:

```
# 3.1 Distribution of final grade
plt.figure(figsize=(7, 4))
sns.histplot(df["G3"], bins=20, kde=True, color="steelblue")
plt.title("Distribution of Final Grade (G3)")
plt.savefig("fig1_g3_distribution.png", dpi=150, bbox_inches="tight")
plt.show()
```

OUTPUT:



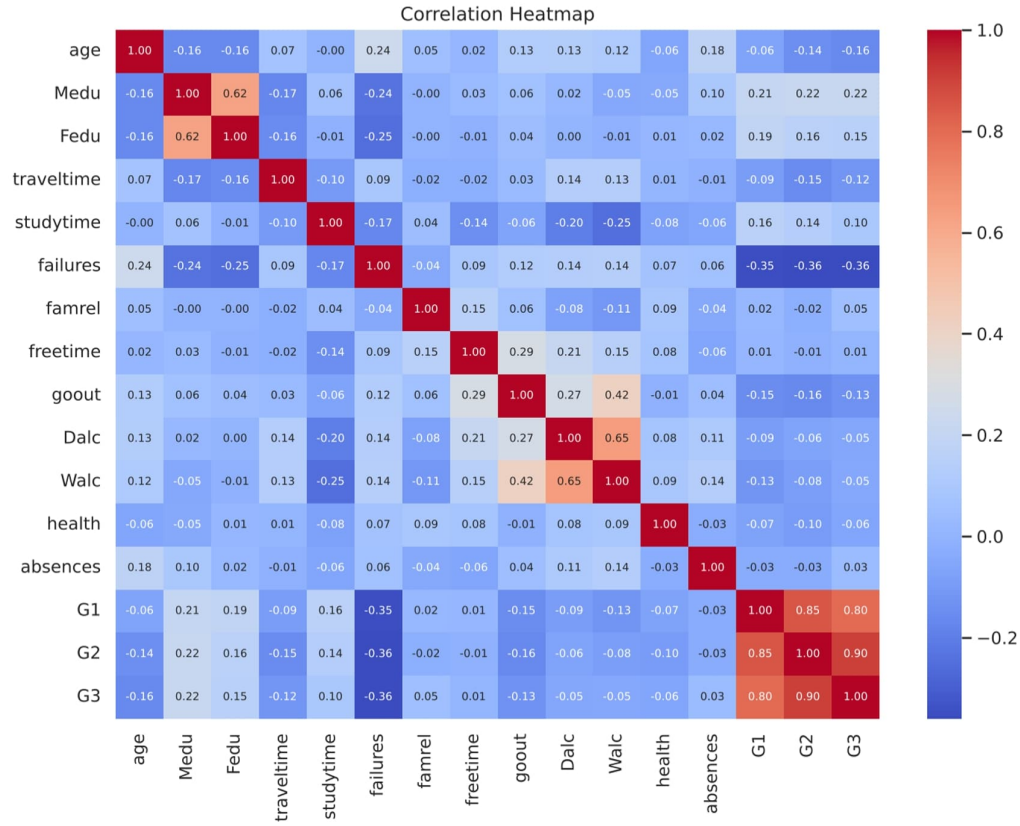
Most students score between about 8 and 16, with the highest count near 10 to 11. A separate bar at 0 holds the 38 students who scored zero, which creates a second small peak on the left of the distribution.

Step 6: Correlation Heatmap

INPUT:

```
# 3.2 Correlation heatmap
num_cols = df.select_dtypes(include=np.number).columns
plt.figure(figsize=(11, 8))
sns.heatmap(df[num_cols].corr(), annot=True, fmt=".2f",
        cmap="coolwarm", annot_kws={"size": 7})
plt.title("Correlation Heatmap")
plt.savefig("fig2_correlation.png", dpi=150, bbox_inches="tight")
plt.show()
print(df[num_cols].corr()["G3"].sort_values(ascending=False))
```

OUTPUT:




```
G3           1.000000
G2           0.904868
G1           0.801468
Medu         0.217147
Fedu         0.152457
studytime    0.097820
famrel       0.051363
absences     0.034247
freetime     0.011307
Walc        -0.051939
Dalc        -0.054660
health      -0.061335
traveltime  -0.117142
goout       -0.132791
age         -0.161579
failures    -0.360415
Name: G3, dtype: float64
```

G2 (0.90) and G1 (0.80) are very strongly correlated with the final grade. Among the other attributes, past failures show the strongest negative relation (-0.36), while mother's education

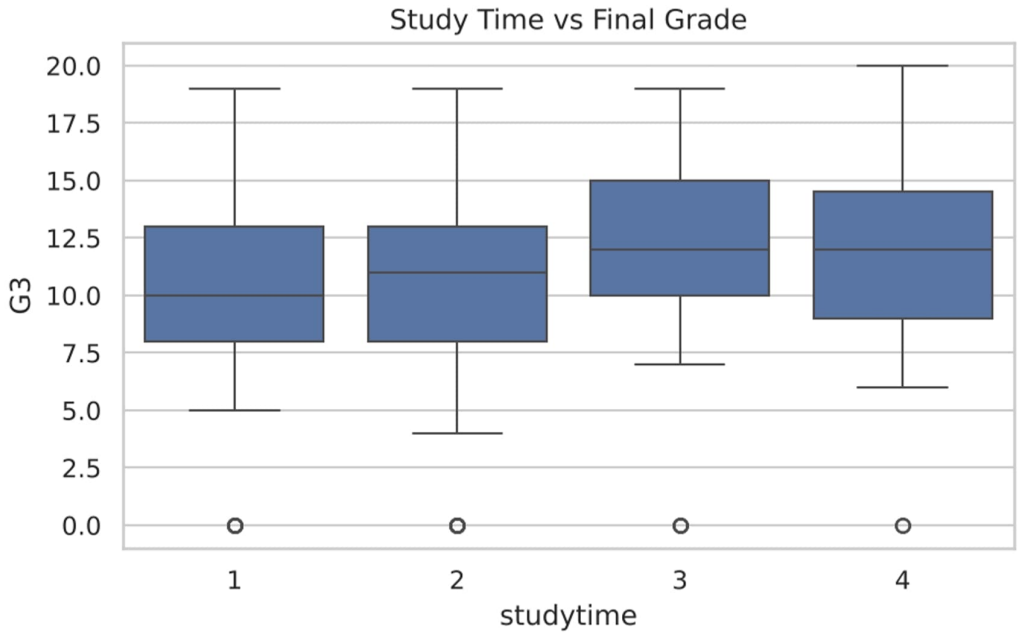
(0.22) and father's education (0.15) are mildly positive. Age (-0.16) and going out (-0.13) are weakly negative, and study time is weakly positive (0.10).

Step 7: Study Time vs Final Grade

INPUT:

```
# 3.3 Study time vs final grade
plt.figure(figsize=(7, 4))
sns.boxplot(x="studytime", y="G3", data=df)
plt.title("Study Time vs Final Grade")
plt.savefig("fig3_studytime.png", dpi=150, bbox_inches="tight")
plt.show()
```

OUTPUT:



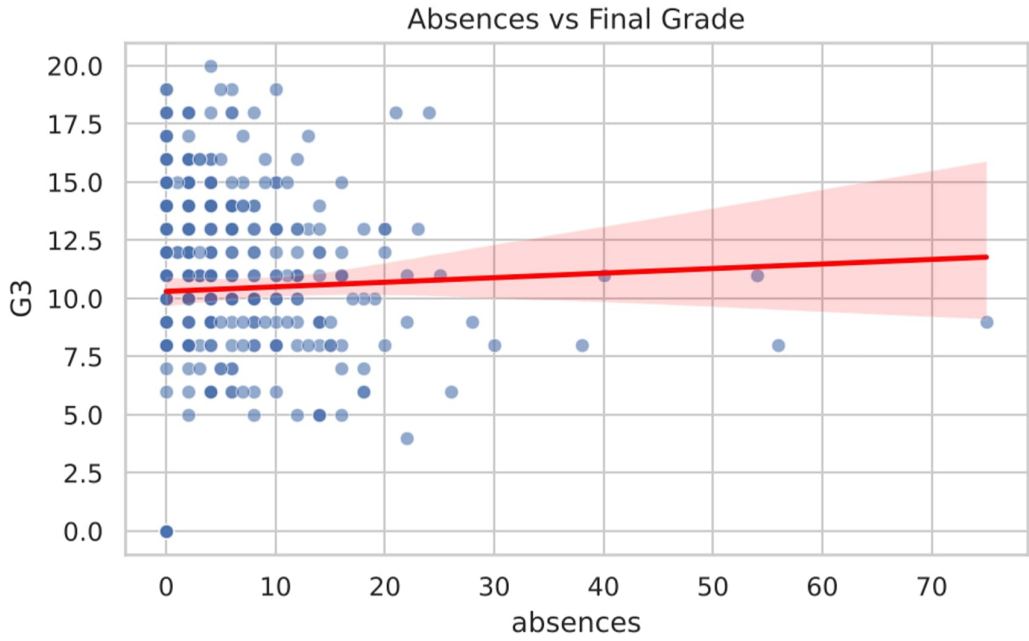
The average final grade is 10.05 and 10.17 for study-time levels 1 and 2, and rises to 11.40 and 11.26 for levels 3 and 4. Students who study more tend to score about one point higher on average, although the ranges overlap heavily.

Step 8: Absences vs Final Grade

INPUT:

```
# 3.4 Absences vs final grade
plt.figure(figsize=(7, 4))
sns.scatterplot(x="absences", y="G3", data=df, alpha=0.6)
sns.regplot(x="absences", y="G3", data=df, scatter=False, color="red")
plt.title("Absences vs Final Grade")
plt.savefig("fig4_absences.png", dpi=150, bbox_inches="tight")
plt.show()
```

OUTPUT:



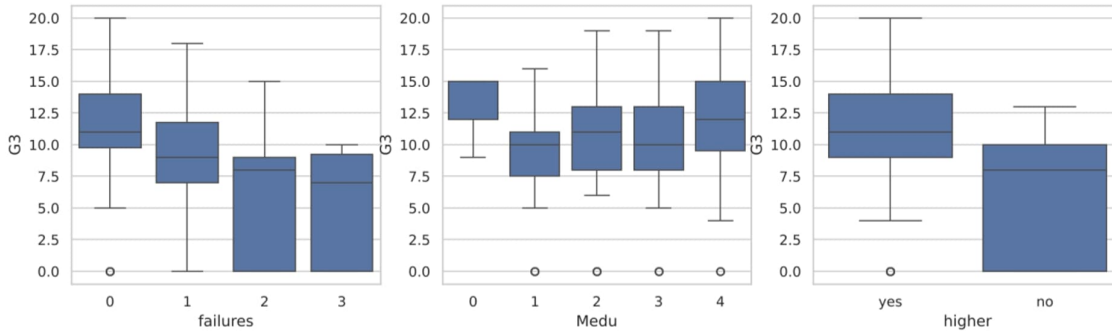
The trend line is almost flat and slightly upward across all students. This is caused by the 38 zero-grade students, who all have 0 absences. When those students are excluded, the relationship turns negative (correlation -0.21), meaning more absences go with lower grades.

Step 9: Failures, Parental Education and Higher-Education Aspiration

INPUT:

```
# 3.5 Past failures, parental education and higher-education aspiration
fig, ax = plt.subplots(1, 3, figsize=(15, 4))
sns.boxplot(x="failures", y="G3", data=df, ax=ax[0])
sns.boxplot(x="Medu", y="G3", data=df, ax=ax[1])
sns.boxplot(x="higher", y="G3", data=df, ax=ax[2])
plt.savefig("fig5_failures_family.png", dpi=150, bbox_inches="tight")
plt.show()
```

OUTPUT:



The average grade falls steadily with past failures: 11.25 for no failures, 8.12 for one, 6.24 for two and 5.69 for three. Students who want higher education score 10.61 on average against 6.80 for those who do not. Mother's education shows a mild positive trend.

Step 10: Average Final Grade by Category

INPUT:

```
# 3.6 Average final grade by category
for col in ["sex", "address", "internet", "schoolsup", "romantic"]:
    print(df.groupby(col)["G3"].mean().round(2), "\n")
```

OUTPUT:

```
sex
F     9.97
M    10.91
Name: G3, dtype: float64

address
R     9.51
U    10.67
Name: G3, dtype: float64

internet
no      9.41
yes    10.62
Name: G3, dtype: float64

schoolsup
no     10.56
yes     9.43
Name: G3, dtype: float64

romantic
no     10.84
yes     9.58
Name: G3, dtype: float64
```

Male students (10.91) average slightly higher than female students (9.97). Students with urban addresses (10.67), internet access (10.62), no extra school support (10.56) and no romantic relationship (10.84) also have higher average grades than their counterparts.

Step 11: Feature Engineering and Selection

INPUT:

```
# New features
```

```
df["avg_alcohol"] = (df["Dalc"] * 5 + df["Walc"] * 2) / 7
df["parent_edu"] = (df["Medu"] + df["Fedu"]) / 2
df["social_index"] = df["goout"] + df["freetime"]

# Two feature sets: with and without the earlier period grades G1, G2
target = "G3"
X_full = pd.get_dummies(df.drop(columns=[target]), drop_first=True)
X_nograde = pd.get_dummies(df.drop(columns=[target, "G1", "G2"]), drop_first=True)
y = df[target]

# Rank features by absolute correlation with G3
corr = (X_nograde.assign(G3=y).corr()["G3"]
        .drop("G3").abs().sort_values(ascending=False))
print("Top 10 features (without G1, G2):")
print(corr.head(10))
```

OUTPUT:

```
Top 10 features (without G1, G2):
failures        0.360415
Medu            0.217147
parent_edu      0.205224
higher_yes      0.182465
age             0.161579
Fedu            0.152457
goout           0.132791
romantic_yes    0.129970
traveltime      0.117142
Mjob_health     0.116158
Name: G3, dtype: float64
```

Without G1 and G2, the features most correlated with G3 are failures (0.36), mother's education (0.22), the engineered parent_edu feature (0.21), wanting higher education (0.18) and age (0.16). The new parent_edu feature ranks third, which shows it carries useful information.

Step 12: Model Development

INPUT:

```
def run_models(X, y, label):
    X_tr, X_te, y_tr, y_te = train_test_split(
        X, y, test_size=0.2, random_state=42)
    scaler = StandardScaler().fit(X_tr)
    Xs_tr, Xs_te = scaler.transform(X_tr), scaler.transform(X_te)

    models = {
        "Linear Regression": LinearRegression(),
        "Ridge": Ridge(alpha=10),
        "Lasso": Lasso(alpha=0.1),
        "Decision Tree": DecisionTreeRegressor(max_depth=5, random_state=42),
        "Random Forest": RandomForestRegressor(n_estimators=300, random_state=42),
```

```
    }
    rows = []
    for name, m in models.items():
        m.fit(Xs_tr, y_tr)
        pred = m.predict(Xs_te)
        cv = cross_val_score(m, scaler.transform(X), y, cv=5, scoring="r2").mean()
        rows.append({
            "Setting": label, "Model": name,
            "MAE": mean_absolute_error(y_te, pred),
            "RMSE": np.sqrt(mean_squared_error(y_te, pred)),
            "R2": r2_score(y_te, pred),
            "CV R2": cv,
        })
    return pd.DataFrame(rows), models, X_te, y_te, scaler

res_full, models_full, Xte_f, yte_f, sc_f = run_models(X_full, y, "With G1, G2")
res_ng, models_ng, Xte_n, yte_n, sc_n = run_models(X_nograde, y, "Without G1, G2")
```

OUTPUT:

This step defines a function that trains Linear Regression, Ridge, Lasso, Decision Tree and Random Forest models on both feature sets and calculates MAE, RMSE, R^2 and cross-validated R^2. It produces no printed output; the results are shown in the next step.

Step 13: Model Comparison

INPUT:

```
results = pd.concat([res_full, res_ng]).round(3)
print(results.to_string(index=False))
results.to_csv("model_results.csv", index=False)
```

OUTPUT:

```
       Setting             Model   MAE  RMSE     R2  CV R2
  With G1, G2 Linear Regression 1.647 2.378  0.724  0.784
  With G1, G2             Ridge 1.626 2.358  0.729  0.787
  With G1, G2             Lasso 1.375 2.161  0.772  0.809
  With G1, G2     Decision Tree 1.350 2.411  0.716  0.717
  With G1, G2     Random Forest 1.174 1.960  0.813  0.835
Without G1, G2 Linear Regression 3.395 4.196  0.141  0.001
Without G1, G2             Ridge 3.381 4.178  0.149  0.019
Without G1, G2             Lasso 3.380 4.183  0.147  0.040
Without G1, G2     Decision Tree 3.644 4.835 -0.140 -0.118
Without G1, G2     Random Forest 3.092 3.882  0.265  0.124
```

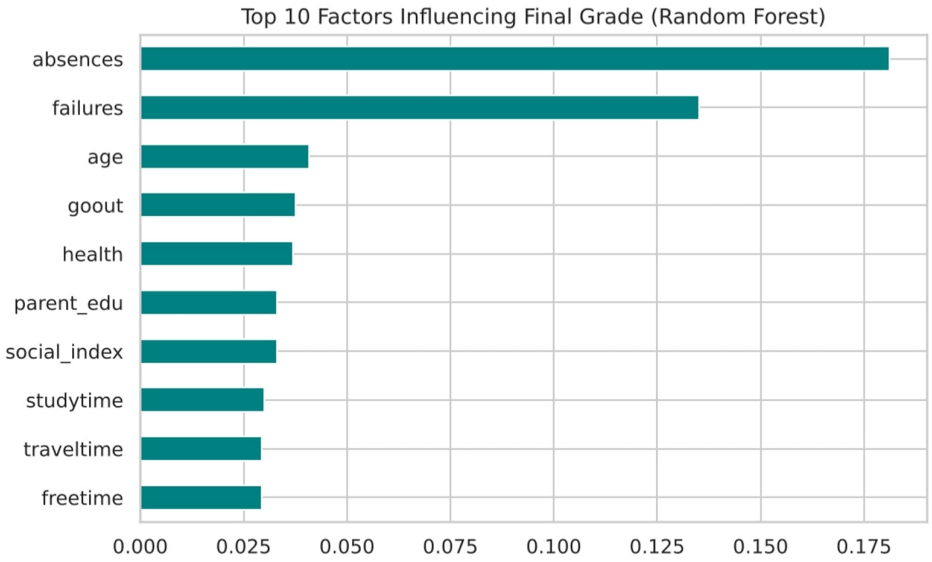
With G1 and G2, Random Forest performs best (MAE 1.17, RMSE 1.96, R^2 0.81, cross-validated R^2 0.84), followed by Lasso (R^2 0.77). Without G1 and G2 all models drop sharply: the best, Random Forest, reaches only R^2 0.27 on the test set and 0.12 in cross-validation, and the Decision Tree falls below zero. This shows that earlier grades are by far the strongest predictor.

Step 14: Top Factors Influencing Final Grade

INPUT:

```
# Top factors influencing final grade (without G1, G2)
rf = RandomForestRegressor(n_estimators=300, random_state=42).fit(X_nograde, y)
imp = (pd.Series(rf.feature_importances_, index=X_nograde.columns)
      .sort_values(ascending=False).head(10))
plt.figure(figsize=(8, 5))
imp[::-1].plot(kind="barh", color="teal")
plt.title("Top 10 Factors Influencing Final Grade (Random Forest)")
plt.savefig("fig6_feature_importance.png", dpi=150, bbox_inches="tight")
plt.show()
```

OUTPUT:



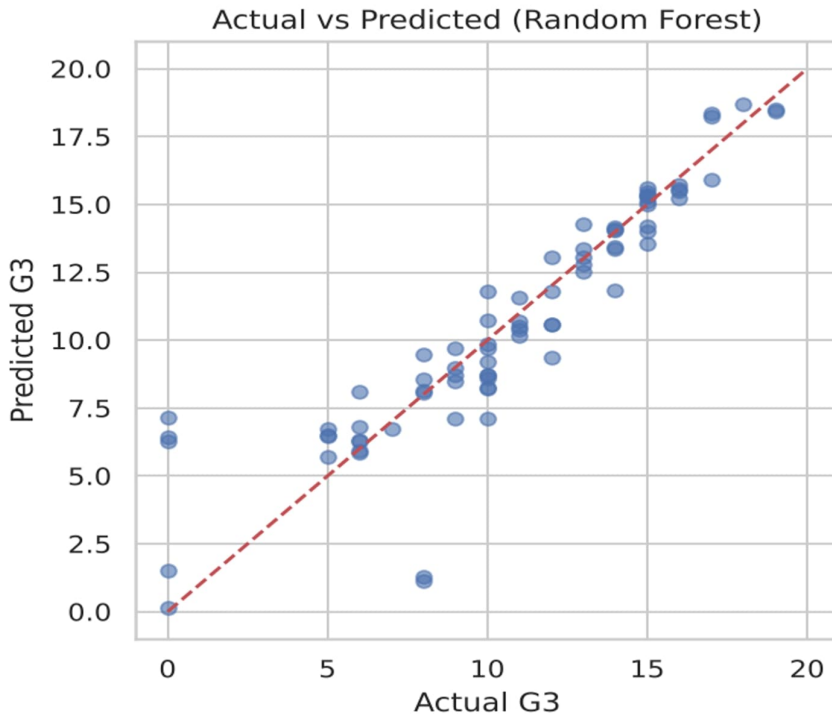
Using Random Forest without G1 and G2, absences (about 0.18) and failures (about 0.13) are the two most important factors, followed by age, going out and health. Absences rank high mainly because all zero-grade students have 0 absences, which the model uses to separate them from other students.

Step 15: Actual vs Predicted Grades

INPUT:

```
# Actual vs predicted (Random Forest, with G1, G2)
best = models_full["Random Forest"]
pred = best.predict(sc_f.transform(Xte_f))
plt.figure(figsize=(5, 5))
plt.scatter(yte_f, pred, alpha=0.6)
plt.plot([0, 20], [0, 20], "r--")
plt.xlabel("Actual G3"); plt.ylabel("Predicted G3")
plt.title("Actual vs Predicted (Random Forest)")
plt.savefig("fig7_actual_vs_pred.png", dpi=150, bbox_inches="tight")
plt.show()
```

OUTPUT :



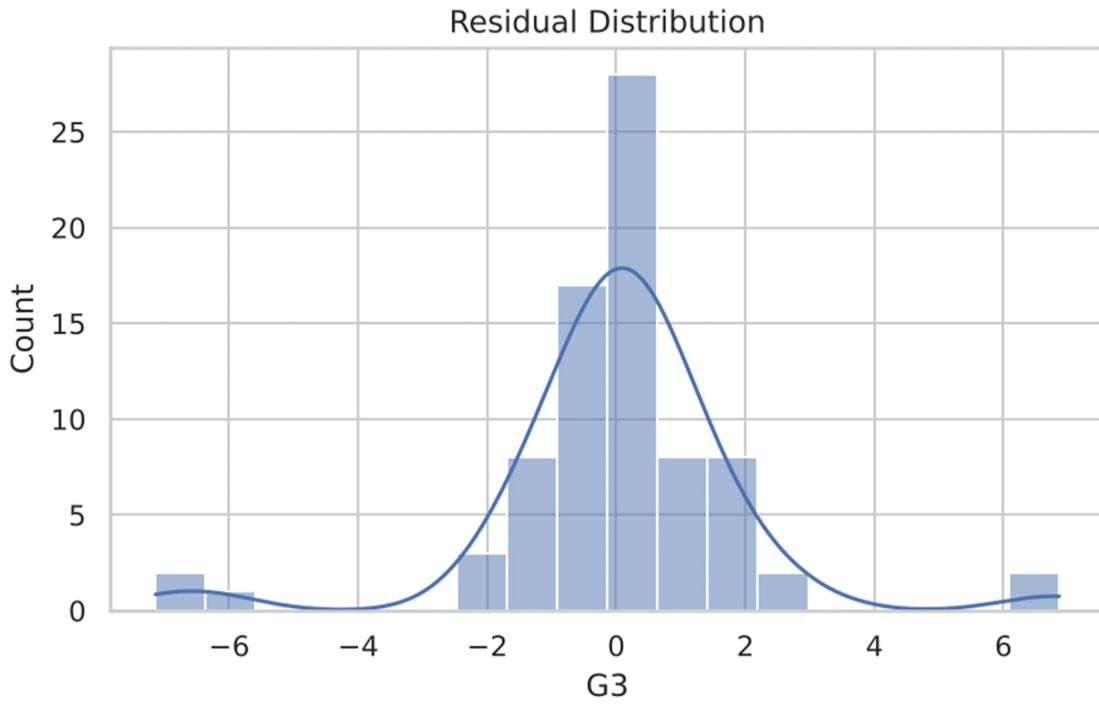
For the best model (Random Forest with G1 and G2), most points lie close to the dashed line, especially for grades from 10 to 19. The main errors are a few students whose actual grade is 0 but were predicted around 6 to 7, and one student with actual grade 8 predicted near 1.

Step 16: Residual Distribution

INPUT:

```
# Residual distribution
plt.figure(figsize=(7, 4))
sns.histplot(yte_f - pred, kde=True)
plt.title("Residual Distribution")
plt.savefig("fig8_residuals.png", dpi=150, bbox_inches="tight")
plt.show()
```

OUTPUT:



The prediction errors are centred near zero and most fall between -2 and +2 grade points. A few large errors of about ±6 to 7 points correspond to the zero-grade cases the model finds hard to predict.

CONCLUSION:

The Student Performance Analysis system was successfully developed using Python and its data-science libraries. The UCI Student Performance Dataset (395 students, 33 attributes) was cleaned, explored through visualizations and used to train five regression models. The analysis shows that the earlier period grades G1 and G2 are the strongest predictors of the final grade, and Random Forest gave the best result with an R^2 of 0.81 and an MAE of 1.17 grade points. Among the other factors, past failures, the wish to pursue higher education and parental education were clearly related to performance (students with no failures averaged 11.25 against 5.69 for three failures), while study time showed only a modest effect. Without G1 and G2 the models explain only a small part of the variation (best R^2 of 0.27), so behavioural and background attributes alone are not enough for accurate grade prediction. Thus, the project helps teachers identify at-risk students early and shows which factors deserve attention.